

RELATÓRIO — SISTEMA DE TELEMETRIA PARA DECOLAGEM

Atividade Integradora — Python, análise de telemetria e decisão de decolagem

1. Objetivo

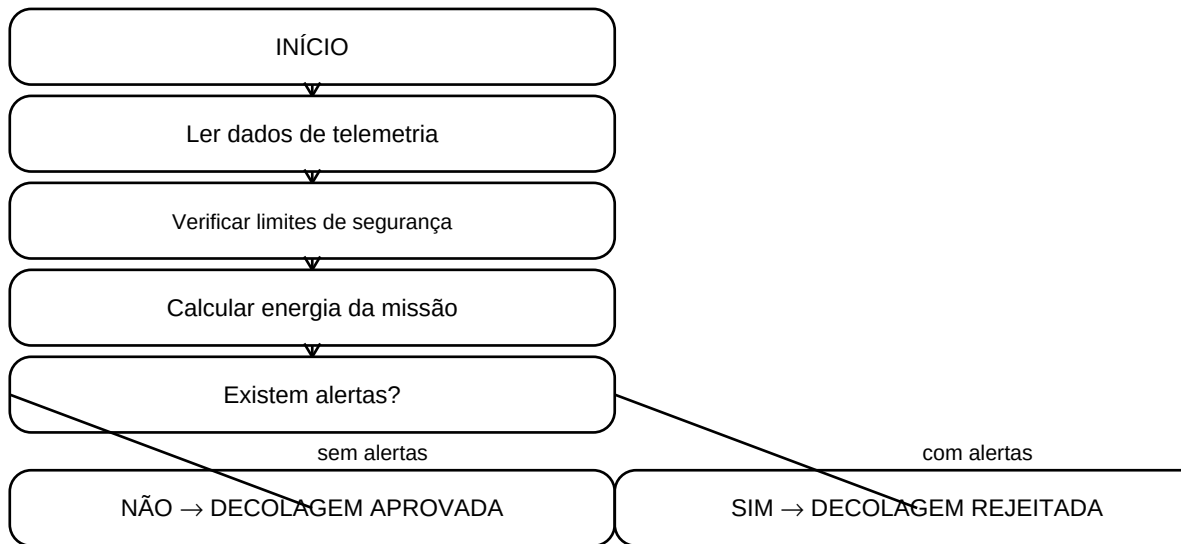
Desenvolver um programa em Python capaz de receber dados de telemetria de uma nave, verificar condições de segurança, realizar uma análise energética e determinar se a decolagem pode ser aprovada.

2. Organização da telemetria

Parâmetro	Critério de segurança	Interpretação
Temperatura interna	10 a 35 °C	Fora da faixa gera alerta
Temperatura externa	20 a 50 °C	Fora da faixa gera alerta
Integridade	0 ou 1	1 = OK; 0 = corrompida
Energia	mínimo 40%	Abaixo do mínimo gera alerta
Pressão	1 a 4 bar	Fora da faixa gera alerta
Módulos críticos	0 ou 1	1 = OK; 0 = falha

3. Algoritmo de verificação

O programa lê os seis parâmetros, cria uma lista de alertas e verifica cada limite. Depois calcula a energia necessária para a missão. A decisão final depende da ausência ou presença de alertas.



4. Pseudocódigo

PSEUDOCÓDIGO — SISTEMA DE TELEMETRIA

INÍCIO

Exibir critérios de segurança

Ler temperatura interna
Ler temperatura externa
Ler integridade
Ler nível de energia
Ler pressão
Ler status dos módulos críticos

Criar lista de alertas vazia

SE temperatura interna < 10 OU > 35
 adicionar alerta
FIMSE

SE temperatura externa < 20 OU > 50
 adicionar alerta
FIMSE

SE integridade = 0
 adicionar alerta
FIMSE

SE energia < 40
 adicionar alerta
FIMSE

SE pressão < 1 OU > 4
 adicionar alerta
FIMSE

SE módulos críticos = 0
 adicionar alerta
FIMSE

consumo_por_hora $\leftarrow$ 10
tempo_missao $\leftarrow$ 4
energia_necessaria $\leftarrow$ consumo_por_hora $\times$ tempo_missao

SE energia disponível $\geq$ energia necessária
 calcular margem
 exibir energia suficiente
SENÃO
 calcular energia faltante
 exibir energia insuficiente
 adicionar alerta
FIMSE

SE lista de alertas estiver vazia
 exibir "DECOLAGEM APROVADA"
SENÃO
 exibir "DECOLAGEM REJEITADA"
 exibir todos os alertas
FIMSE

FIM

5. Análise energética

O consumo considerado é de 10% por hora e a duração da missão é de 4 horas. Portanto: $10 \times 4 = 40\%$ de energia necessária. O programa compara a energia disponível com esse valor, informando a margem quando suficiente ou a quantidade faltante quando insuficiente.

6. Lógica de decisão

A decolagem somente é aprovada quando a lista de alertas permanece vazia. Qualquer temperatura fora do intervalo, integridade igual a 0, energia abaixo de 40%, pressão fora de 1–4 bar, módulo crítico igual a 0 ou energia insuficiente para a missão provoca rejeição.

7. Testes de execução

Teste 1 — cenário aprovado: temperatura interna 25 °C; externa 30 °C; integridade 1; energia 80%; pressão 2,5 bar; módulos críticos 1.

Resultado esperado: DECOLAGEM APROVADA.

Teste 2 — cenário rejeitado: temperatura interna 5 °C; externa 55 °C; integridade 0; energia 30%; pressão 5 bar; módulos críticos 0.

Resultado esperado: DECOLAGEM REJEITADA, com alertas dos parâmetros fora dos limites e energia insuficiente para a missão.

8. Código-fonte

Dados de telemetria

```
print("===== CRITERIOS PARA DECOLAGEM =====")
print("Temperatura interna: 10 a 35 °C")
print("Temperatura externa: 20 a 50 °C")
print("Integridade: 0 ou 1")
print("Energia: mínimo 40%")
print("Pressão: 1 a 4 bar")
print("Módulos críticos: 0 ou 1")
print("=====")
```

```
print("===== DIGITE OS VALORES PEDIDOS =====")
print("=====")
```

```
def obter_telemetria():
    return {
        'temperatura': temperatura,
        'temperatura_externa': temperatura_externa,
        'integridade': integridade,
        'nivel_energia': energia,
        'pressao': pressao,
        'status_modulos_criticos': status_modulos_criticos,
    }
```

TEMPERATURA

```
while True:
    try:
        temperatura = float(input("Digite a Temperatura interna da Nave: "))
        if temperatura >= 0:
            break
        else:
            print("A temperatura não pode ser negativa.")
    except ValueError:
        print("Insira um número válido!!!")
```

TEMPERATURA EXTERNA

```
while True:
    try:
        temperatura_externa = float(input("Digite a temperatura externa da nave: "))
        if temperatura_externa >= 0:
            break
        else:
            print("Temperatura negativa.")
    except ValueError:
        print("Insira um número válido!!!")
```

INTEGRIDADE

```
while True:
    try:
        integridade = int(input("Digite a integridade estrutural (0 ou 1): "))
        if integridade == 0 or integridade == 1:
            break
        else:
            print("Integridade deve ser 0 ou 1.")
    except ValueError:
        print("Insira um número válido!!!")
```

ENERGIA

```
while True:
    try:
        energia = int(input("Digite o Nivel de Energia (%): "))
```

```
if 0 <= energia <= 100:
    break
else:
    print("Energia deve estar entre 0 a 100")
except ValueError:
    print("Insira um número válido!!!")
```

PRESSÃO

```
while True:
    try:
        pressao = float(input("Digite a pressão dos tanques (bar): "))
        if pressao >= 0:
            break
        else:
            print("A pressão precisa ser positiva!!!")
    except ValueError:
        print("Insira um número válido!!!")
```

MÓDULOS CRÍTICOS

```
while True:
    try:
        status_modulos_criticos = int(
            input("Digite o status dos módulos críticos (1 para OK, 0 para falha): ")
        )
        if status_modulos_criticos in [0, 1]:
            break
        else:
            print("Status dos módulos críticos deve ser 0 ou 1.")
    except ValueError:
        print("Insira um número válido!!!")
```

VERIFICAÇÃO

```
def verificar_decolagem(telemetria):
    alertas = []

    if telemetria['temperatura'] > 35:
        alertas.append("Alerta: Temperatura interna acima do limite seguro!")

    if telemetria['temperatura'] < 10:
        alertas.append("Alerta: Temperatura interna abaixo do limite seguro!")

    if telemetria['temperatura_externa'] < 20:
        alertas.append("Alerta: Temperatura externa abaixo do limite seguro!")

    if telemetria['temperatura_externa'] > 50:
        alertas.append("Alerta: Temperatura externa acima do limite seguro!")

    if telemetria['integridade'] == 0:
        alertas.append("Alerta: Integridade estrutural corrompida!")

    if telemetria['nivel_energia'] < 40:
        alertas.append("Alerta: Nível de energia crítico!")

    if telemetria['pressao'] < 1:
        alertas.append("Alerta: Pressão abaixo do limite seguro!")

    if telemetria['pressao'] > 4:
        alertas.append("Alerta: Pressão acima do limite seguro!")

    if telemetria['status_modulos_criticos'] == 0:
        alertas.append("Alerta: Falha em módulos críticos!")

    consumo_por_hora = 10
    tempo_missao = 4
    energia_necessaria = consumo_por_hora * tempo_missao
```

```

print("\n===== ANÁLISE ENERGÉTICA =====")
print(f"Consumo por hora: {consumo_por_hora}%")
print(f"Tempo da missão: {tempo_missao} horas")
print(f"Energia necessária: {energia_necessaria}%")
print(f"Energia disponível: {telemetria[nivel_energia]}%")

if telemetria[nivel_energia] >= energia_necessaria:
    margem = telemetria[nivel_energia] - energia_necessaria
    print(f"Margem de energia: {margem}%")
    print("ANÁLISE: Energia suficiente para a missão.")
else:
    falta = energia_necessaria - telemetria[nivel_energia]
    print(f"Energia faltante: {falta}%")
    print("ANÁLISE: Energia insuficiente para a missão.")
    alertas.append("Energia insuficiente para completar a missão!")

if len(alertas) == 0:
    return "Todos os sistemas estão operando dentro dos parâmetros normais, decolagem aprovada."

resultado = "DECOLAGEM REJEITADA!\n"
resultado += "ALERTAS ENCONTRADOS:\n"

for alerta in alertas:
    resultado += f"- {alerta}\n"

return resultado

telemetria = obter_telemetria()
print("=====")

resultado = verificar_decolagem(telemetria)

print("\n===== RESULTADO DA TELEMETRIA =====")
print("=====")
print(f"> temperatura: {telemetria[temperatura]} °C <")
print(f"> temperatura externa: {telemetria[temperatura_externa]} °C <")
print(f"> integridade: {telemetria[integridade]} <")
print(f"> nível de energia: {telemetria[nivel_energia]} % <")
print(f"> pressão: {telemetria[pressao]} bar <")
print(f"> status dos módulos críticos: {telemetria[status_modulos_criticos]} <")
print("=====")
print(f"RESULTADO DA TELEMETRIA: {resultado}")

```